

The `arty.xdc` File, Explained Line by Line

`final_8` project · Digilent Arty A7-35T · includes the July 16–17 debug notes

Lucas Zhou

July 17, 2026

Contents

1	What Is an XDC File in the First Place?	2
2	Anatomy of One Constraint Line	2
3	The Current, Working XDC — Every Line	3
3.1	Lines 1–16: the header comments	4
3.2	Lines 18–22: the four switch inputs	4
3.3	Lines 24–32: the seven segment outputs	4
3.4	Lines 34–36: the digit select	5
4	The Old Versions, and Why Each Failed	5
4.1	Version 1 (July 13): straight-through guess	5
4.2	Version 2 (July 17, first fix): permuted, still JA-only	5
4.3	What the photo revealed (the two real bugs)	6
4.4	Version 3 (current): why it works	6
5	Debug Notes / Timeline (July 16–17)	7

1 What Is an XDC File in the First Place?

Your `top.v` describes *logic*: “there are inputs called A2, A1, B2, B1, an output bus called `seg`, and here is the truth table connecting them.” Notice that nothing in the Verilog says *where* those signals physically live. The FPGA chip has 324 metal balls on its underside, and the Arty board routes some of them to switches, some to LEDs, some to the Pmod connectors on the edge.

The **XDC file** (*Xilinx Design Constraints*) is the missing link. It tells the place-and-route tool (`nextpnr-xilinx` in our flow):

“The Verilog port named A2 must come out on the physical pin named A10, using 3.3 V signaling.”

That is essentially *all* our XDC does — eleven sentences of that form, one per port. XDC files are written in the **Tcl** scripting language (pronounced “tickle”), which is why the syntax looks nothing like Verilog. You never write algorithms in it here; every line is the same one command with different arguments.

Key mental model: the XDC is a *patch cable list*. Changing it never changes your logic — it only changes which physical pin each logical wire pops out of. That is why every hardware bug in this project was fixed in the XDC while your `top.v` stayed untouched.

2 Anatomy of One Constraint Line

Every functional line in the file is this single pattern:

```
set_property -dict { PACKAGE_PIN A10 IOSTANDARD LVCMOS33 } [get_ports { A2 }];
```

Reading it piece by piece:

set_property The Tcl command. It means “attach the following properties to the following object.” Same command on every line.

-dict { ... } “Here comes a *dictionary* of property–value pairs.” A dictionary is just an alternating list: *name value name value*. Ours contains two pairs:

- **PACKAGE_PIN A10** — property **PACKAGE_PIN** gets value **A10**. This is the physical ball on the chip package. Pin names like **A10**, **G13**, **E15** are grid coordinates on the 324-ball package (row letter + column number), *not* related to your signal names. It is pure coincidence that a switch pin is called **A10** and your port is called **A2**.
- **IOSTANDARD LVCMOS33** — property **IOSTANDARD** gets value **LVCMOS33**, meaning “Low-Voltage CMOS, 3.3 V.” FPGA pins can speak many electrical dialects (1.8 V, 2.5 V, differential pairs, ...). Everything on the Arty’s switches and Pmod connectors is wired for 3.3 V, so every line says **LVCMOS33**. Omitting it is an error in Vivado and bad practice everywhere.

[get_ports { A2 }] In Tcl, square brackets mean “run this inner command first and substitute its result” — like $f(g(x))$ in math. **get_ports** looks up a port *by the name it has in the top-level Verilog module*. So this fetches your **input wire A2** and hands it to **set_property**. The name must match the Verilog port exactly; for a bus you name one bit at a time: **seg[0]**, **seg[1]**, ...

; End of statement. (In Tcl a newline also ends a statement, so the semicolon is optional — it is kept for clarity.)

comment Anything after # is a comment. Double ## is just a style convention, same as one.

So the whole line reads, in English: *“Find the top-level port named A2, solder it (logically) to package ball A10, and drive it with 3.3 V CMOS levels.”*

3 The Current, Working XDC — Every Line

```
1 ## final_8: 4 switches in, seg[6:0] out to Pmod SSD straddling JA and JB
2 ##
3 ## The Pmod SSD has two 6-pin headers: J1 (AA-AD, GND, VCC) plugs into the
4 ## TOP ROW of JA, and J2 (AE, AF, AG, CAT, GND, VCC) plugs into the TOP ROW
5 ## of JB. So four of the signals live on JB pins 1-4, not JA's bottom row.
6 ##
7 ## The display glyphs are mounted 180 deg rotated as viewed on the Arty
8 ## (decimal points sit top-left), so each module signal lights the segment
9 ## diametrically opposite its name: AA = viewed bottom, AD = viewed top,
10 ## AB = viewed lower-left, etc. The assignments below bake in both the
11 ## JA/JB split and that rotation.
12 ##
13 ## top.v numbers segments 1-7 bottom-to-top (seg[0..6] = viewed D,C,E,G,B,F,A).
14 ## If a segment still lights in the wrong position, permute these pin
15 ## assignments rather than the top.v table.
16
17 ## Switches (A2=SW3, A1=SW2, B2=SW1, B1=SW0)
18 set_property -dict { PACKAGE_PIN A10 IOSTANDARD LVCMOS33 } [get_ports { A2 }];
19 set_property -dict { PACKAGE_PIN C10 IOSTANDARD LVCMOS33 } [get_ports { A1 }];
20 set_property -dict { PACKAGE_PIN C11 IOSTANDARD LVCMOS33 } [get_ports { B2 }];
21 set_property -dict { PACKAGE_PIN A8 IOSTANDARD LVCMOS33 } [get_ports { B1 }];
22
23 ## Seven-segment outputs (viewed position -> module signal -> FPGA pin)
24 set_property -dict { PACKAGE_PIN G13 IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; #
    viewed D (bottom) -> AA -> JA1
25 set_property -dict { PACKAGE_PIN E16 IOSTANDARD LVCMOS33 } [get_ports { seg[1] }]; #
    viewed C (lower-right) -> AF -> JB2
26 set_property -dict { PACKAGE_PIN B11 IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; #
    viewed E (lower-left) -> AB -> JA2
27 set_property -dict { PACKAGE_PIN D15 IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; #
    viewed G (middle) -> AG -> JB3
28 set_property -dict { PACKAGE_PIN E15 IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; #
    viewed B (upper-right) -> AE -> JB1
29 set_property -dict { PACKAGE_PIN A11 IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; #
    viewed F (upper-left) -> AC -> JA3
30 set_property -dict { PACKAGE_PIN D12 IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; #
    viewed A (top) -> AD -> JA4
31
32 ## Pmod SSD digit select (CAT on the module's J2 pin 4 = JB4, NOT JA10):
33 ## picks which of the two digits is lit. top.v drives it constant.
34 set_property -dict { PACKAGE_PIN C15 IOSTANDARD LVCMOS33 } [get_ports { C }];
```

3.1 Lines 1–16: the header comments

Pure documentation — the tool ignores them. They record the two hardware facts discovered on July 16 (Section 4) so the next reader does not rediscover them the hard way.

3.2 Lines 18–22: the four switch inputs

One line per input port. Each maps a Verilog input to one of the four slide switches at the bottom edge of the board:

Line	Verilog port	Package pin	Board part
19	A2 (high bit of A)	A10	switch SW3
20	A1 (low bit of A)	C10	switch SW2
21	B2 (high bit of B)	C11	switch SW1
22	B1 (low bit of B)	A8	switch SW0

Where do pins like A10 come from? From the board schematic — Digilent publishes a “master XDC” listing which package pin every switch, LED and connector wire uses. Our project keeps a condensed copy in `docs/hardware-reference.md`. These four lines have been correct from day one and never changed.

3.3 Lines 24–32: the seven segment outputs

One line per bit of output `reg [6:0] seg`. Three name spaces meet on each line, and the comments spell out all three:

1. **Your numbering** (`seg[0]..seg[6]`): the problem’s “segment 1 to 7,” which counts the segments *bottom-to-top*: bottom bar, lower-right, lower-left, middle, upper-right, upper-left, top. This ordering was reverse-engineered from your own case table (e.g. your pattern for “1” sets exactly bits 1 and 4 $\Rightarrow$ those must be the two right-hand verticals).
2. **Module signal names** (`AA..AG`): what the Pmod SSD’s silkscreen calls its seven inputs.
3. **FPGA package pins** (`G13, E16, ...`): where the Arty routes each Pmod connector pin.

Line by line:

Line	Port	Meaning in <code>top.v</code>	SSD signal	Pin (connector)
25	<code>seg[0]</code>	segment 1 = bottom bar	AA	G13 (JA pin 1)
26	<code>seg[1]</code>	segment 2 = lower-right	AF	E16 (JB pin 2)
27	<code>seg[2]</code>	segment 3 = lower-left	AB	B11 (JA pin 2)
28	<code>seg[3]</code>	segment 4 = middle bar	AG	D15 (JB pin 3)
29	<code>seg[4]</code>	segment 5 = upper-right	AE	E15 (JB pin 1)
30	<code>seg[5]</code>	segment 6 = upper-left	AC	A11 (JA pin 3)
31	<code>seg[6]</code>	segment 7 = top bar	AD	D12 (JA pin 4)

Why the scrambled-looking middle column? Two physical quirks, both explained in Section 4: the module spans *two* connectors (JA and JB), and its glyphs are mounted upside down, so signal AA lights the *bottom* segment as you view it, AD the top, and so on.

3.4 Lines 34–36: the digit select

The Pmod SSD contains *two* digits sharing the seven segment wires. The CAT input chooses which digit is actually lit (in normal use you would toggle it fast to show two different numbers; we hold it constant from assign C = 1'b1 in top.v, so one digit is always on). It lives on JB pin 4 = package pin C15.

4 The Old Versions, and Why Each Failed

The XDC went through three versions. The Verilog logic was correct the whole time — the July 13 simulation sweep of all 16 inputs matched the case table exactly, and it still does. Every failure below is purely about *which physical pin each signal came out of*.

4.1 Version 1 (July 13): straight-through guess

```
1 ## Seven-segment outputs on Pmod JA
2 set_property -dict { PACKAGE_PIN G13 IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; # JA1
   -> AA
3 set_property -dict { PACKAGE_PIN B11 IOSTANDARD LVCMOS33 } [get_ports { seg[1] }]; # JA2
   -> AB
4 set_property -dict { PACKAGE_PIN A11 IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; # JA3
   -> AC
5 set_property -dict { PACKAGE_PIN D12 IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; # JA4
   -> AD
6 set_property -dict { PACKAGE_PIN D13 IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; # JA7
   -> AE
7 set_property -dict { PACKAGE_PIN B18 IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; # JA8
   -> AF
8 set_property -dict { PACKAGE_PIN A18 IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; # JA9
   -> AG
9 set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports { C }]; # JA10
```

This assumed (a) `seg[N]` simply means SSD signal number $N+1$ in alphabetical order AA→AG, and (b) the whole module plugs into connector JA alone, using both its rows.

Symptom on hardware: garbage shapes — a “7”-looking glyph, disconnected fragments. **Wrong because** your `top.v` numbers segments geometrically (bottom-to-top), not alphabetically, so each lit segment landed in an unrelated position.

4.2 Version 2 (July 17, first fix): permuted, still JA-only

```
1 set_property -dict { PACKAGE_PIN D12 IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; # JA4
   -> AD
2 set_property -dict { PACKAGE_PIN A11 IOSTANDARD LVCMOS33 } [get_ports { seg[1] }]; # JA3
   -> AC
3 set_property -dict { PACKAGE_PIN D13 IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; # JA7
   -> AE
4 set_property -dict { PACKAGE_PIN A18 IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; # JA9
   -> AG
5 set_property -dict { PACKAGE_PIN B11 IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; # JA2
   -> AB
```

```

6 set_property -dict { PACKAGE_PIN B18 IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; # JA8
  -> AF
7 set_property -dict { PACKAGE_PIN G13 IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; # JA1
  -> AA

```

This version fixed the *numbering* mismatch: decoding the case table showed `seg[0..6]` must mean segments D, C, E, G, B, F, A (the bottom-to-top order). Assuming the SSD’s names matched the standard seven-segment letters and everything sat on JA, this permutation should have produced correct digits.

Symptom on hardware: still partial digits — for input 1011 (2×3), only three segments of the expected six lit up. **Wrong because** both physical assumptions from Version 1 were still baked in, and both turned out to be false.

4.3 What the photo revealed (the two real bugs)

Photo IMG_8616 (July 17), input 1011, expected “6”:

1. **The module straddles two connectors.** The Pmod SSD has two separate 6-pin headers: J1 (AA AB AC AD GND VCC) in the top row of JA, and J2 (AE AF AG CAT GND VCC) in the top row of JB. Versions 1–2 drove AE/AF/AG/CAT on JA’s *bottom row* (pins D13 B18 A18 K16) — pins the module does not even touch. Segments E, F, G and the digit select were floating: three of your seven wires went nowhere. That is exactly “part of a number.”
2. **The glyphs are mounted 180° rotated.** The decimal points sit at the *top-left* of each digit (normally bottom-right) — the giveaway that the LED package is upside down relative to its labels. The lit segments confirmed it: the FPGA was driving signals AA, AC, AD high, and the photo showed *bottom, upper-left, top* lit — each one diametrically opposite its textbook position. Uncorrected, every digit would render upside down (a 6 would read as a 9).

A useful detail: the three segments that *did* light were a perfectly correct half of a “6.” That proved the logic and the Version-2 permutation were right as far as JA reached, and isolated the fault to the unwired JB half plus the rotation.

4.4 Version 3 (current): why it works

The final XDC applies two corrections on top of Version 2:

- **JA/JB split:** AE→E15 (JB1), AF→E16 (JB2), AG→D15 (JB3), CAT→C15 (JB4). Now all eight module inputs are actually driven.
- **180° rotation:** swap each signal with its opposite (AA↔AD, AB↔AE, AC↔AF; AG is the middle bar and maps to itself). So “viewed top” is driven via AD, “viewed bottom” via AA, etc.

Sanity check with input 1011 ($2 \times 3 = 6$): `top.v` outputs `seg = 1101111`, i.e. every segment on except `seg[4]` (upper-right) — which is precisely the shape of a 6, now rendered complete and upright.

5 Debug Notes / Timeline (July 16–17)

1. **Simulation** (Icarus Verilog): all 16 input combinations produce **seg** patterns matching the case table — consistent with the 2-bit $\times$ 2-bit multiplier (results 0,1,2,3,4,6,9). Logic verified; simulation cannot catch pin-mapping bugs.
2. **Build + flash, XDC v1**: clean build, but the display showed scrambled shapes. Diagnosis: segment-numbering mismatch (geometric vs. alphabetical). Fix: permute pins (v2).
3. **Build + flash, XDC v2**: still partial digits. 1011 gave three segments of a six.
4. **Photo analysis** (IMG_8616): revealed the JA+JB straddle (three segments + digit select never wired) and the 180°-rotated glyphs (decimal points top-left; each lit segment opposite its signal name).
5. **Build + flash, XDC v3**: all seven segments and the digit select now driven, rotation corrected. Awaiting visual confirmation; if the wrong digit of the two is lit, flip **assign C = 1'b1** to **1'b0** in **top.v** and rebuild.

Lessons for next time:

- Simulation checks logic; only hardware (or a datasheet) checks pinout. They fail independently.
- When a peripheral misbehaves, first ask “is every wire actually connected to something?” Floating pins produce exactly the “partial” symptom seen here.
- Fix mapping problems in the XDC, not by rewriting a correct truth table. The Verilog never changed after July 13.
- A photo of a known input is a powerful debug tool: three lit segments were enough to deduce both remaining bugs.